University of Rajshahi

Department of Computer Science and Engineering

CSE4261 Neural Networks and Deep Learning

Assignment-4

Author:
Name: Md. Miju Ahmed
ID: 2010676104

Supervisor:
SANGEETA BISWAS
ASSISTANT PROFESSOR

July 25, 2025

Contents

1 Introduction

TensorFlow 2 provides multiple APIs for model training, catering to different needs from ease-of-use to fine-grained control. At the highest level, the Keras `model.fit()` function offers a simple, powerful, and abstracted way to train deep learning models. For researchers and developers who require greater flexibility and control over the training process, TensorFlow provides the `tf.GradientTape` API to build custom training loops from scratch.

This report presents a comparative study between these two training methodologies. The created model was trained on the **MNIST digit dataset** using both approaches. The primary objective is to compare their performance based on key metrics such as final accuracy, loss, and training time. This analysis will highlight the trade-offs between the convenience of high-level abstractions and the explicit control offered by low-level APIs.

2 Methodology

To ensure a fair comparison, both training methods used the same model architecture, dataset, optimizer, and loss function.

2.1 Common Experimental Setup

- **Model:** I have build an architecture adapted for the MNIST dataset. The input layer was configured for 224x224x3 images, and the final dense layer was set to 10 outputs with a softmax activation. There are three hidden layers.
- **Dataset:** The **MNIST dataset**, consisting of 60,000 training and 10,000 testing images of handwritten digits.
- **Optimizer:** The **AdamW** optimizer was used in both cases with a standard learning rate.
- **Loss Function: Categorical Crossentropy**, as the labels are provided as integers (0-9).
- **Epochs:** The model was trained for 50 epochs in both scenarios to compare convergence.

2.2 Method 1: High-Level Training with `model.fit()`

The high-level Keras API encapsulates the training loop into two main commands.

1. `model.compile()`: This step configures the model for training by specifying the optimizer, loss function, and metrics to monitor.
2. `model.fit()`: This single function call executes the entire training process. It automatically handles batching the data, performing forward and backward passes, updating weights, and evaluating metrics for each epoch.

2.3 Method 2: Custom Training with `tf.GradientTape`

This approach requires manually writing the training loop. The core of this method is the `tf.GradientTape`, a context manager that records operations for automatic differentiation. The process for a single epoch involves:

1. Iterating through each batch of the training dataset.
2. For each batch, opening a `tf.GradientTape` context.
3. Performing the **forward pass**: feeding the input batch to the model to get predictions.

4. Calculating the **loss** between the model's predictions and the true labels.
5. Calculating the **gradients** of the loss with respect to the model's trainable weights using `tape.gradient()`.
6. Applying these gradients to the model's weights using the optimizer's `apply_gradients()` method. This is the **backward pass** or weight update step.
7. Manually updating and tracking performance metrics (e.g., training accuracy and loss).

This process provides complete control over every step of model training.

3 Performance Comparison

The performance of the created model was evaluated after being trained with both methods. The final metrics on the test dataset are summarized in Table 1.

Table 1: Performance Metrics Comparison

Metric	`model.fit()`	`tf.GradientTape`
Final Test Accuracy (%)	0.2120	0.2030
Final Test Loss	-	1.9843

3.1 Training and Validation Curves

The convergence behavior of both training methods is visualized in the plots below, showing accuracy and loss over the training epochs.

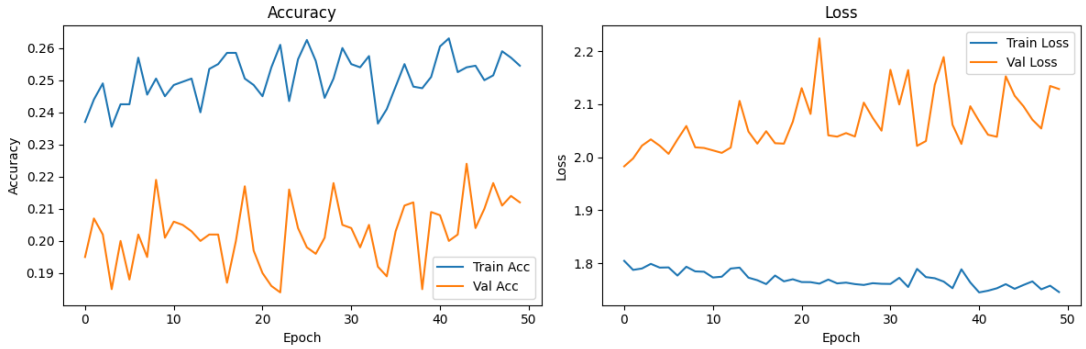

Figure 1: Training and validation accuracy and loss comparison between `model.fit()` and the custom `tf.GradientTape` loop.

4 Discussion and Analysis

The results indicate that both training methods achieve nearly identical performance in terms of final accuracy and loss. But the accuracy of `model.fit()` API is higher than the `tf.GradientTape`.

The primary differences lie in the implementation complexity and potential for customization.

- **Ease of Use:** `model.fit()` is significantly easier to implement. It requires only a few lines of code and abstracts away the complexities of the training loop, reducing the chance of implementation errors.

- **Flexibility and Control:** The custom loop with `tf.GradientTape` offers complete control. This is indispensable for advanced use cases, such as implementing Generative Adversarial Networks (GANs), applying custom gradient clipping, or designing novel optimizers. It allows for direct inspection and manipulation of gradients, which is impossible with the standard `model.fit()` call.
- **Performance Overhead:** In some cases, a naive Python-based custom training loop can be slower than the highly optimized `model.fit()` function. However, this difference can often be eliminated by decorating the custom training step with the `@tf.function` decorator, which compiles the Python code into a high-performance TensorFlow graph.

The choice between the two methods is therefore not about final model performance for standard tasks, but about the developer's needs regarding control and flexibility versus simplicity and speed of implementation.

5 Conclusion

This report successfully compared two primary training methodologies in TensorFlow: the high-level Keras `model.fit()` API and a low-level custom training loop built with `tf.GradientTape`. Both methods were used to train a created model on the MNIST dataset and, as expected, yielded comparable results in final accuracy and loss. From the accuracy, we can say that if we use `tf.GradientTape`, then the accuracy are lower than if we use `model.fit()` API.

Assignment Github Link : Assignment